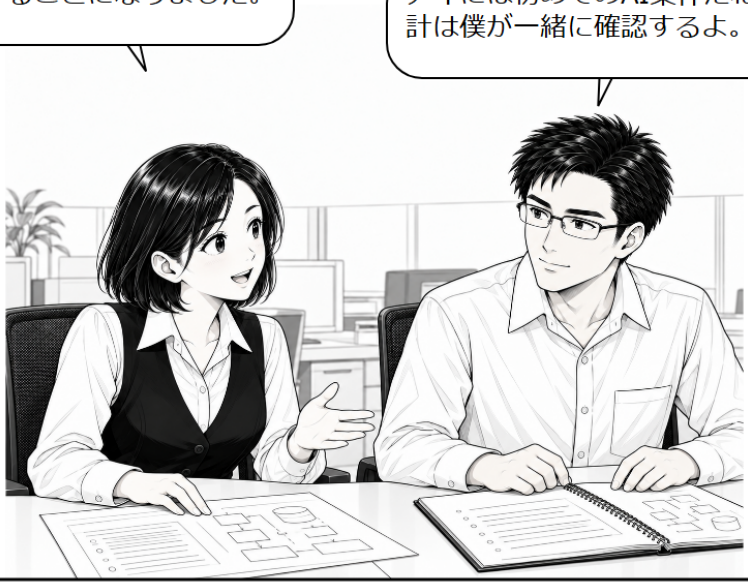
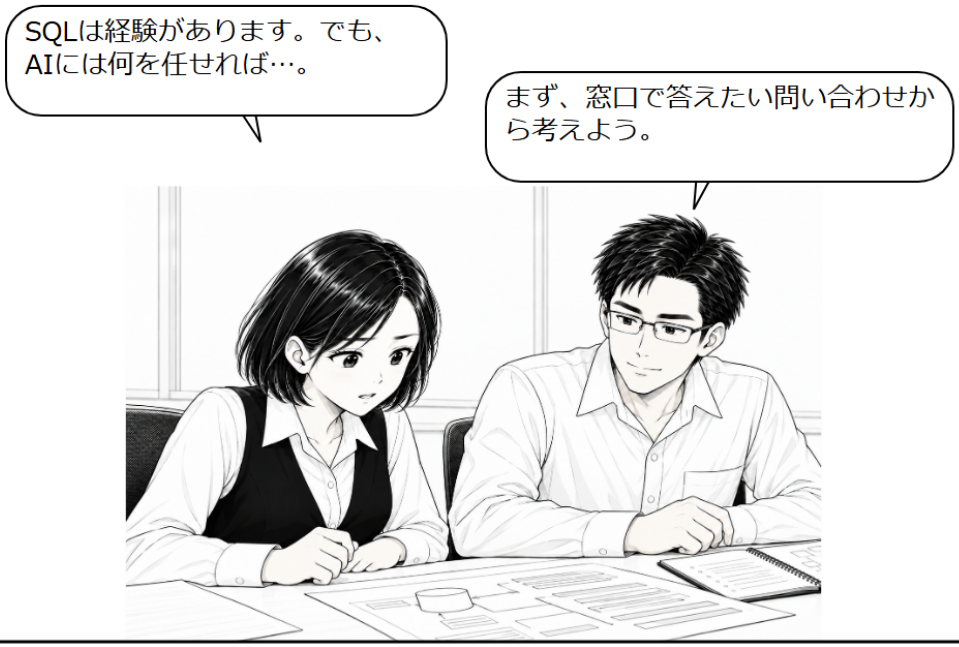
はじめに ― 荒川裕二の技術記事から





想定する問い合わせ

DEV001のサーバーで管理画面が開きません。
確認手順を知りたいです。

現地保守も頼めますか？

手順の根拠と、訪問できる条件を確認したいんですね。

社内の最新手順と、この機器の契約が必要だね。

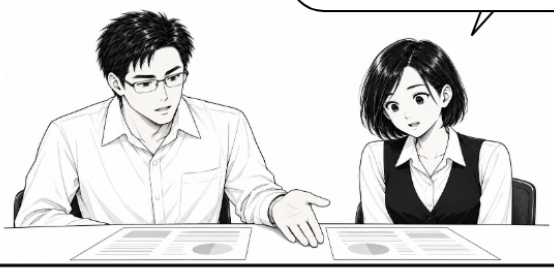
問い合わせへの対応案

アイ
訪問の自動予約は今回のスコープ外です。次の機能拡張で対応予定です。

1. 文書の意味検索と業務データの照会を組み合わせる

手順を探す問いと、契約の条件を確かめる問いだね。

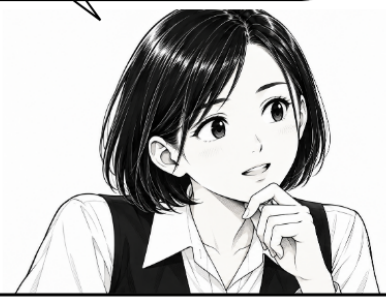
手順は文章。契約は顧客や機器ごとの表ですね。



手順書：非構造化データ　　契約表：構造化データ

神宮寺
文書は意味の近さで探す。契約は表の値を条件と照合する。

ベクトル検索とリレーショナル表の検索を、組み合わせるんですね。

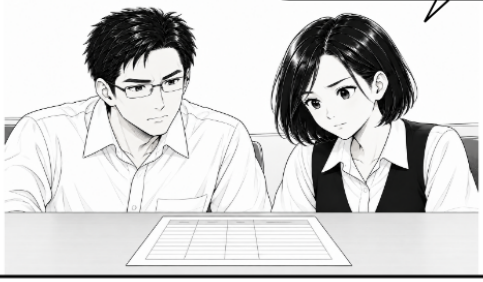


「開かない」「固まった」「応答しない」。言い方が違っても探したい。

文字の一致だけでなく、意味の近さも使うんですね。

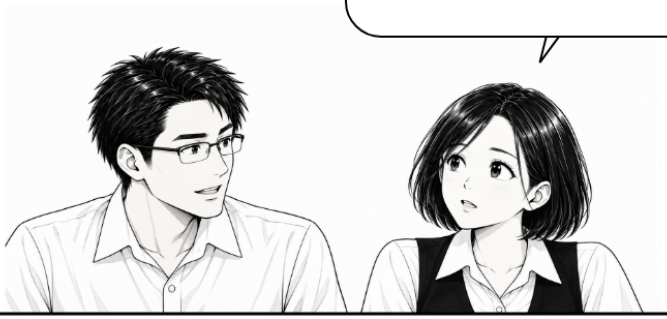
でも、契約の判定が毎回揺らいだら困るよね。

同じ条件なら、いつも同じ結果にしたいです。



いつも同じ基準で判定するには、どうしたらいいかな？

LLMへの指示を、もっと明確にすればいいのでしょうか？



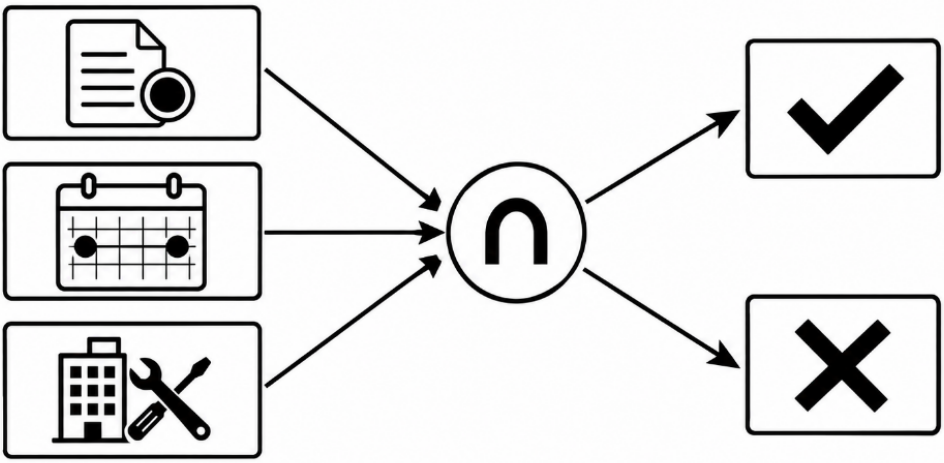
LLMは間違えることがあるし、応答にも揺らぎがある。

契約条件のように厳密さが必要な判定は、任せきりにできないんだ。

プログラムで確定的に判定させるんですね。Pythonなどですか？

そう。DBのビューに、CASE式で判定列を作る方法もある。

ビューに契約の判定列を作る



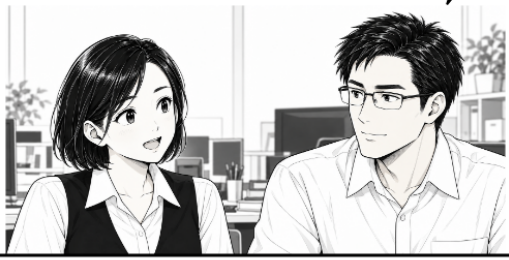
```
CREATE VIEW contract_judgement AS
SELECT c.*,
CASE
WHEN c.status = 'ACTIVE'
AND r.as_of_date
BETWEEN c.start_date
AND c.end_date
AND c.service_scope = 'ONSITE'
THEN 'Y' ELSE 'N'
END AS onsite_eligible
FROM contracts c
CROSS JOIN today r;
```

説明用：contractsは契約表。

todayは「今日の日付」を1行だけ持つ表。

ビューにすると、どんなメリットがありますか？

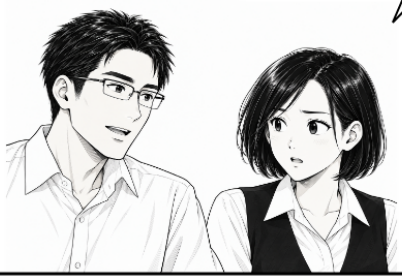
ルールの適用をLLMから切り離せる。同じ契約データと日付なら、結果も同じだ。

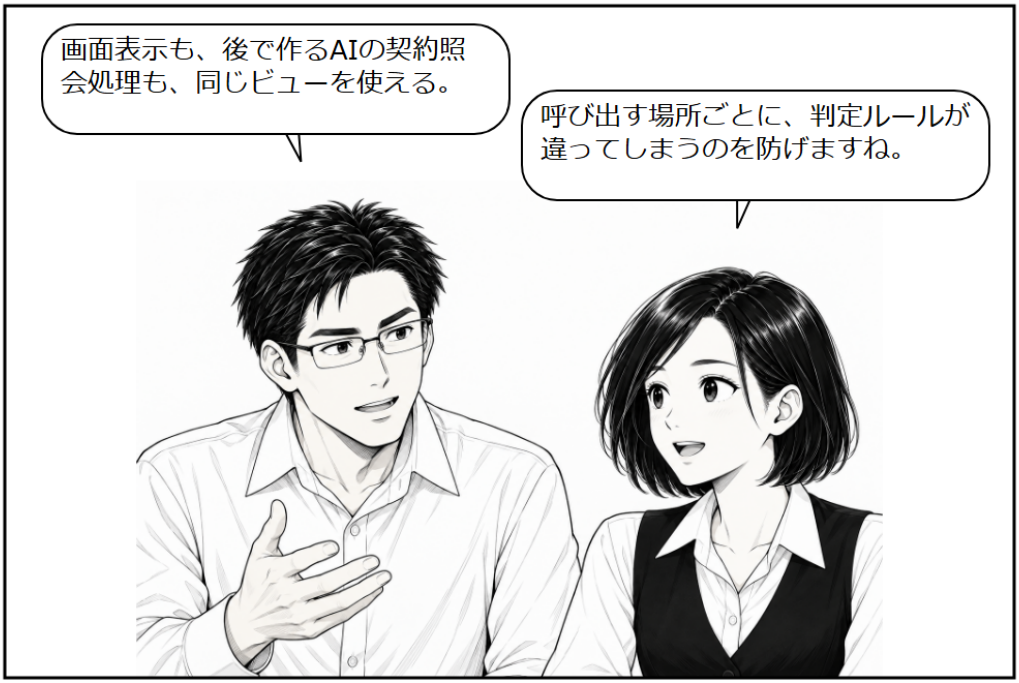


「この契約は対象かな」と毎回推論させず、DBの判定値を使うんですね。

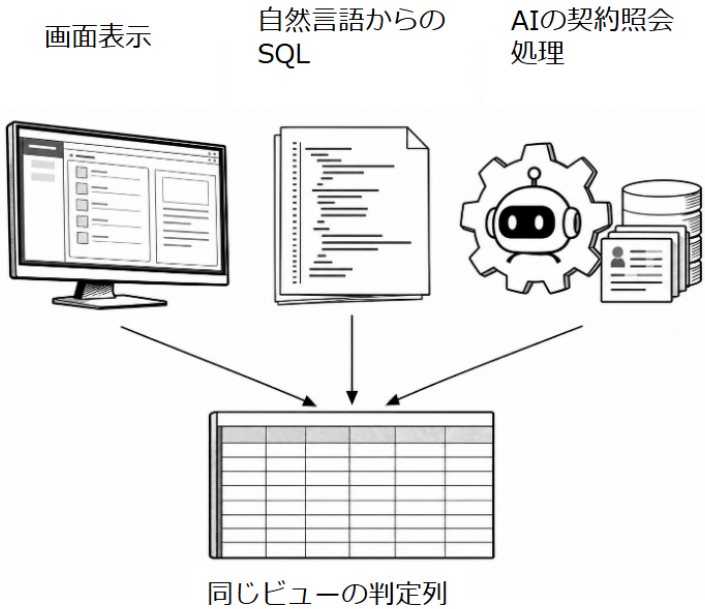
自然言語からSQLを作るときも、三条件を毎回組み立て直さずに済む。

「現地保守の対象」を、判定列のYと対応付ければいいんですね。



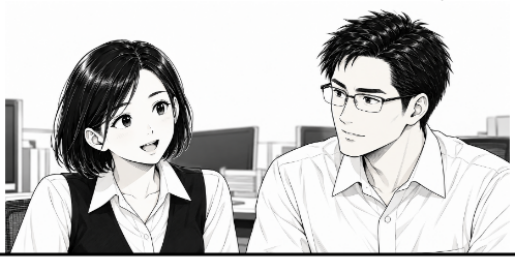


一つの業務ルールを共通で使う



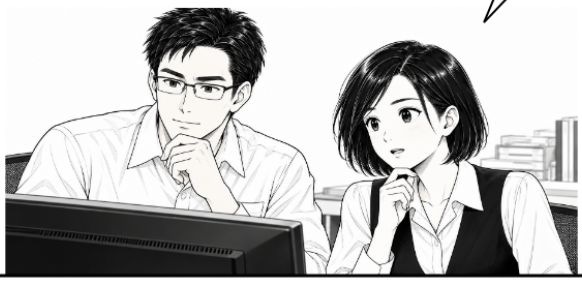
対象外の契約は、ビューから消すんですか？

行は残して、判定列をNにする。状態や期間、サービス範囲も調べられるよ。



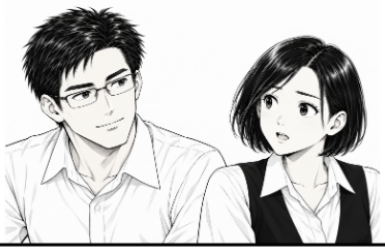
たとえば契約が有効でも、REMOTEなら現地保守は対象外だ。

契約があること、有効なこと、現地保守を使えることは別なんですね。



こういう仕組みも、VIEW as a Harnessと呼べるかもしれないね。

ハーネスって何ですか？



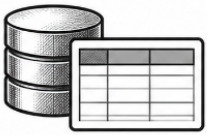
LLMの外側で、使えるデータや道具、実行や検査を支える仕組みだよ。

業務の判定ルールは、その仕組みのコードで一貫して適用する。

意味の理解と、ルールの適用を分ける

LLM：意図や意味を解釈する

ハーネス：決めた業務ルールを適用する

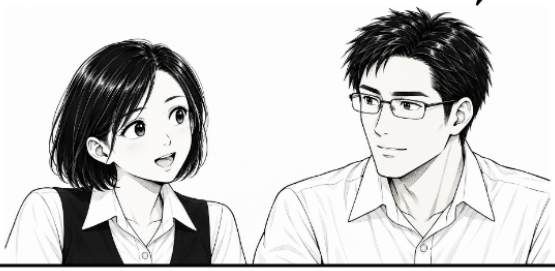


DBのビュー・コード：契約の判定値を返す

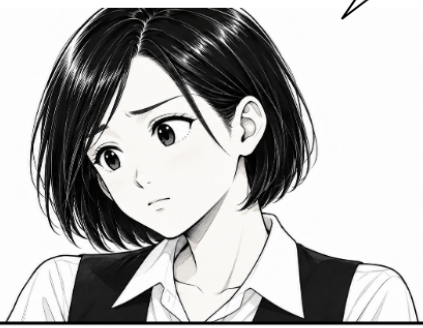
実行側：回答や処理を判定値と照合する

業務上の確定的な判断をLLMから切り離し、ハーネスで守るんですね。

その通り。回答が判定値に沿うかの検査や、権限の設計も必要だ。

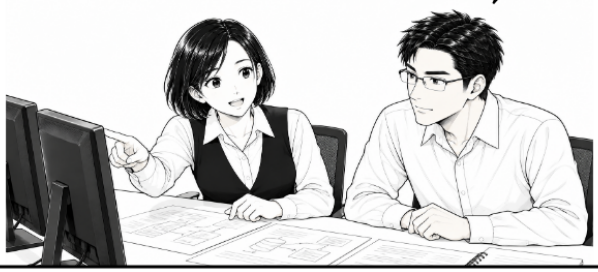


文書データと契約データは、どこで管理したらいいのかな？

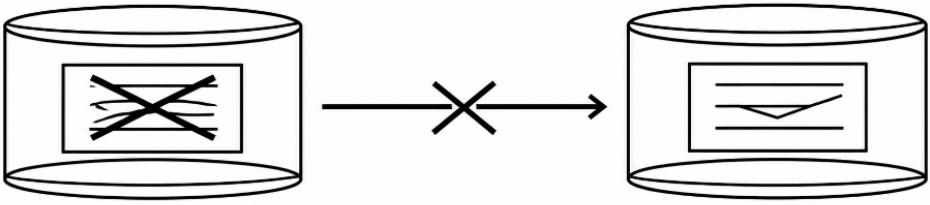


文書はベクトルDB、契約は業務DBに置く方法は？

検索を独立して拡張できる。ただ、変更や閲覧範囲の同期が要る。



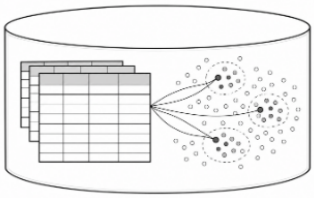
複製の更新が遅れると



業務DB：取消済み　　検索用の複製：未反映

アイ
解約や公開取消が伝わらなければ、案内を間違えますね。

Oracle AI Databaseなら、
業務表とベクトルを同じDBで扱える。



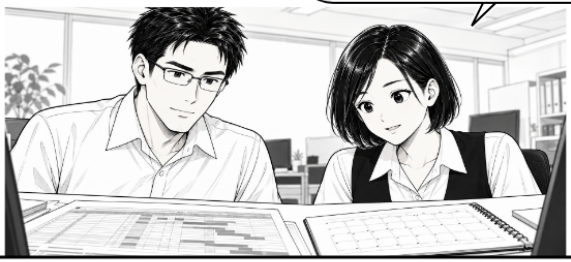
アイ
JOINやEXISTSを使えば、検索用タグへ複製しなくても、業務表の条件を使えるんですね。

鉄道会社を想定した分析例

表と文書に、位置情報も加えられる。鉄道会社を例に見てみよう。

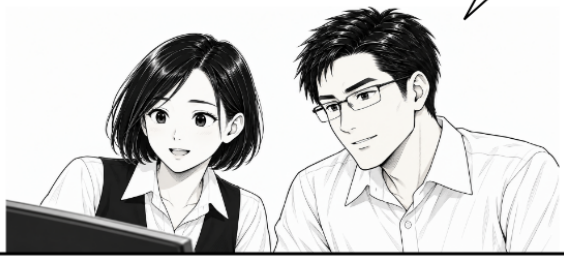
車両タイプと期間は、構造化データのリレーショナル表で絞る。

たとえばタイプ001で、2023年以降の情報を見るんですね。



その期間に新潟県へ進入したかは、どう調べますか？

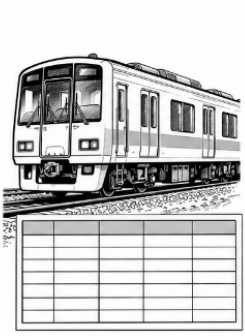
走行位置と県の領域を、地理空間情報として照合する。



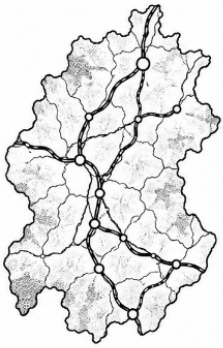
2023年以降の報告で、本文に「ブレーキ」か「制動機」があるかは、全文検索。

その中で、凍結の影響に意味が近い報告を探すのはベクトル検索だ。

一つの分析で多様なデータを扱う

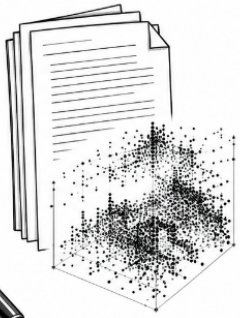


表：
タイプ・日時



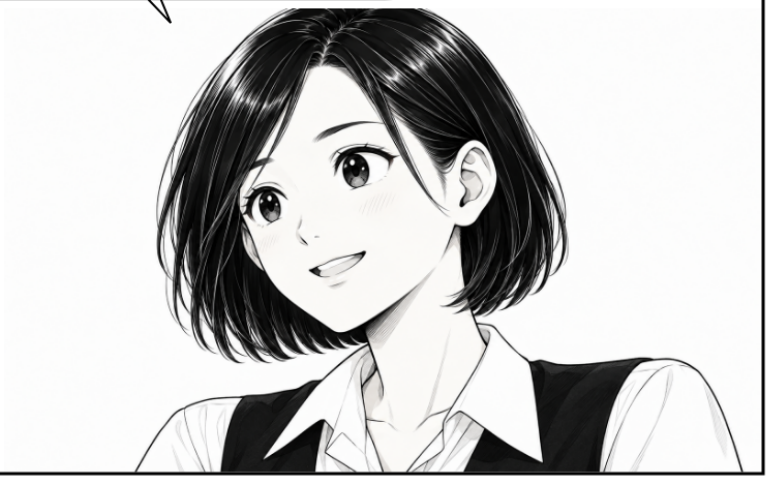
地理空間：
県への進入

全文：
本文の語句



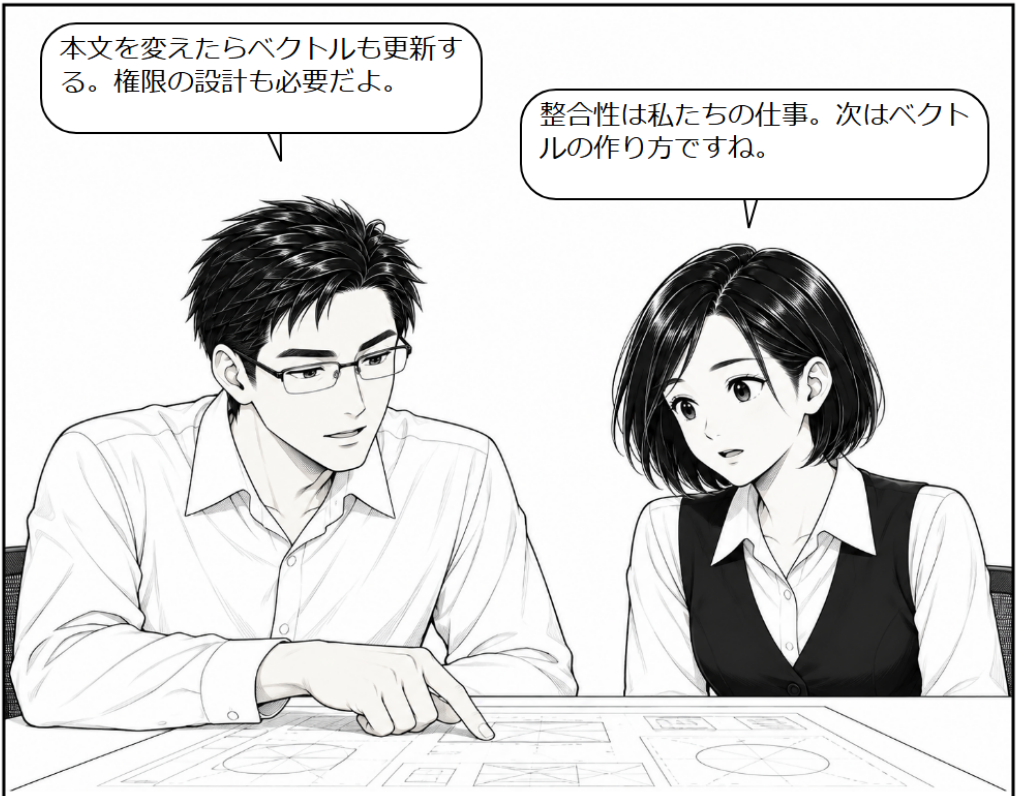
ベクトル：
意味の近さ

表、地図、文章を別々に探して突き合わせなくても、同じDBのSQLで組み合わせられるんですね。



閲覧制限や現地対応判断には最新の契約情報を使いたいですね。

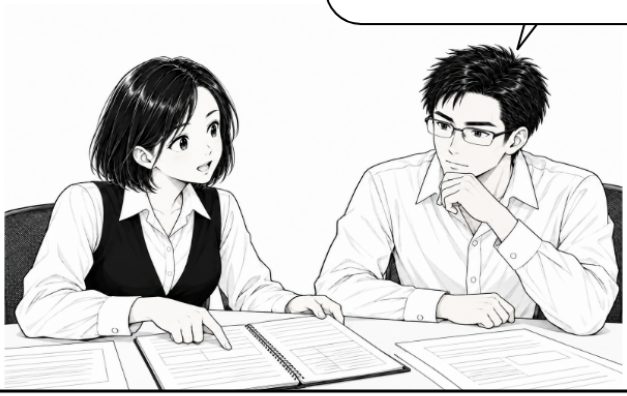
今回はOracle AI Databaseを選びます。


本文を変えたらベクトルも更新する。権限の設計も必要だよ。
整合性は私たちの仕事。次はベクトルの作り方ですね。

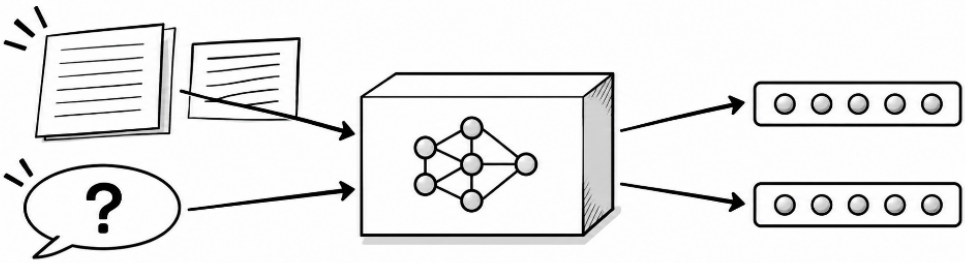
2. 埋め込みモデルをDB内へ持ち込み、DBの中でベクトルを作る

文書をベクトルにするには、何を使うんですか？

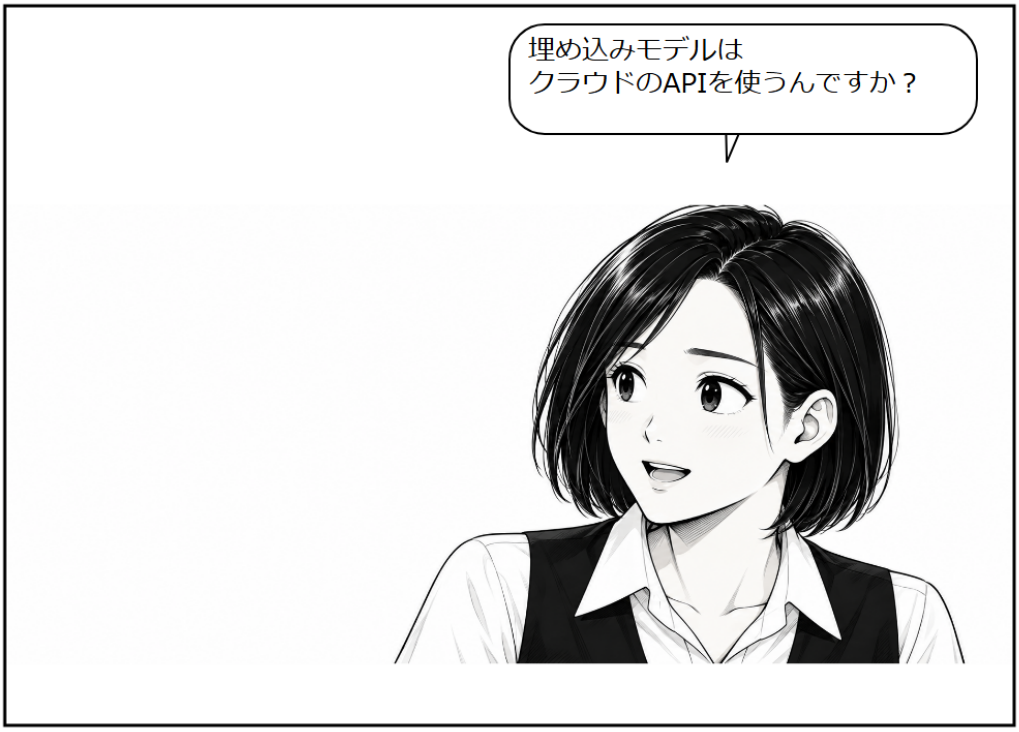
埋め込みモデルだよ。文書と質問を同じモデルで変換する。

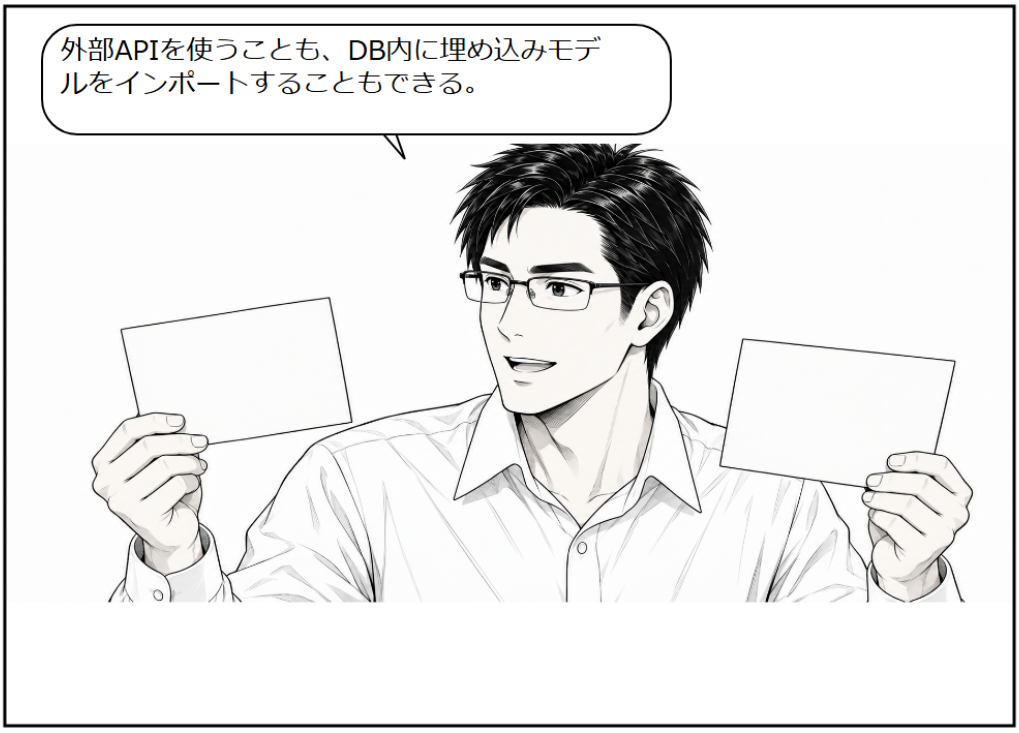


埋め込みモデルとベクトル検索

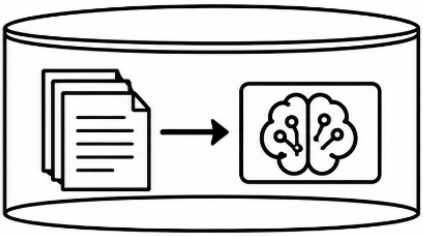


アイ
質問文のベクトルに距離が近いベクトルを持つ文書を見つけるのがベクトル検索ですね

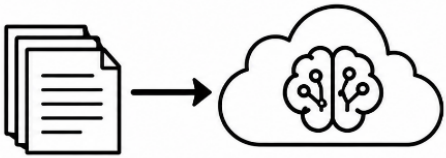

埋め込みモデルは
クラウドのAPIを使うんですか？


外部APIを使うことも、DB内に埋め込みモデルをインポートすることもできる。

埋め込みモデルを動かす場所



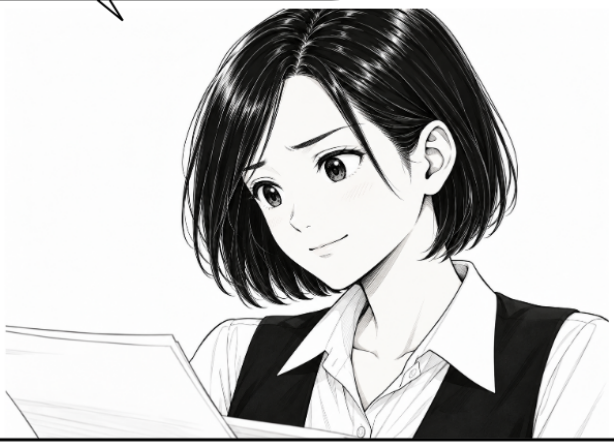
DB内



クラウドなどの外部API

神宮寺
文書のDB登録時には、検索対象の全文書を埋め込みモデルへ送る必要がある。

今回は、検索準備で社内文書の全量を外へ送りたくない。DB内が合いそうです。



ベクトル変換の速度やＤＢ負荷や同時実行数はしっかり検証しよう。

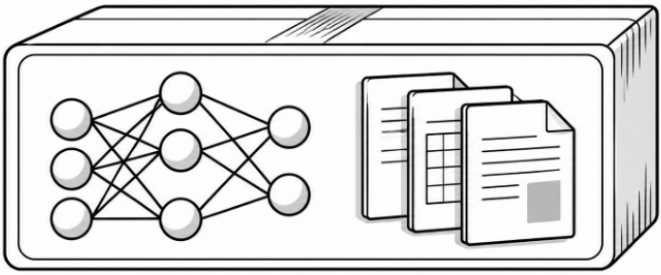
埋め込みモデルはＤＢ内、回答生成LLMはクラウドAPI

場面	埋め込みモデルへ渡すデータ	LLM へ渡すデータ
初回の埋め込み生成	検索対象文書の全チャンク	なし
文書の追加・改訂	追加・変更したチャンク	なし
質問の埋め込み生成	質問文	なし
RAGの回答生成	なし	質問と取得した文書抜粋

必要な抜粋の送信は許可されています。埋め込みはDB内にします。

モデルの品質・利用条件と、DB資源も確認しよう。

multilingual-e5-large



神宮寺
日本語対応の著名なOSS埋め込みモデルの
multilingual-e5-large を
試してみてはどうかな。

神宮寺
ＤＢにインポートするにはこれを ONNX形式に変換しておく

埋め込みモデルのインポート

必要なファイル一式

Object Storage

DBへロード

神宮寺
認証と権限を準備してロードする。付属ファイルも忘れずに。

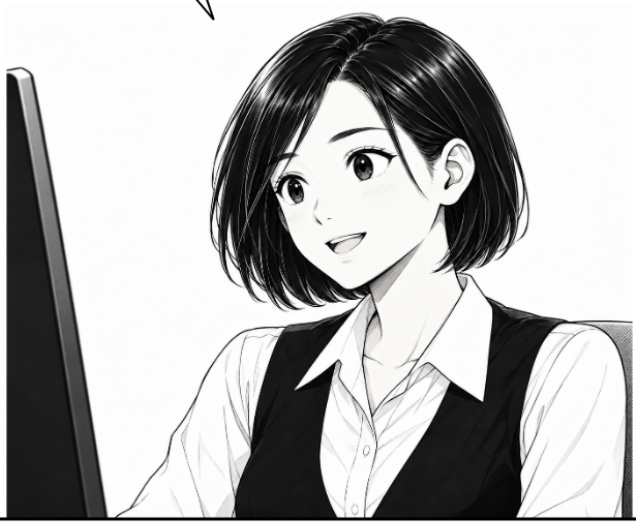
これで質問文に似ている文書の検索は解決ですね？

では、似ている文書を見つければ案内に使えるかな？

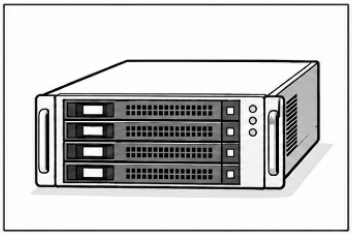
3. チケットの顧客と機種を絞ってベクトル検索する

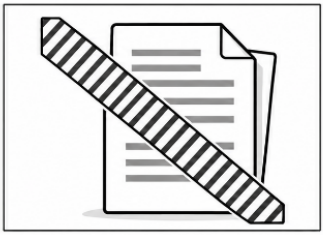
設計の検証ケース：意味だけで検索

「管理画面が開かない」に近い文章なら、候補は見つかりそうです。



対象AX-100／候補に別機種BX-200・廃止済み文書





アイ
あっ、別機種。それに廃止済み。これでは案内に使えません。

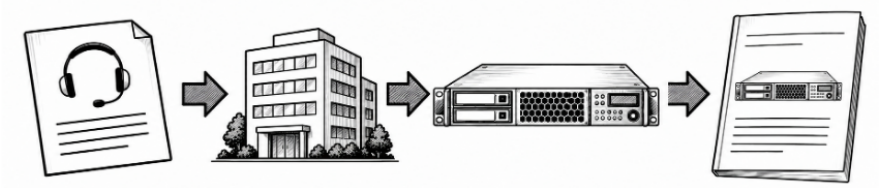
別機種や廃止済みだけでなく、別顧客専用の文書も混ぜてはいけない。

同じ機種の似た症状でも、他社専用の手順を案内したら困りますね。

今回はSR001の顧客C001への回答だ。別顧客の機器や契約も混ぜない。

担当者が他の顧客を扱えても、1回の回答は対応中の顧客に限るんですね。

対応中のチケットから検索対象を決める



SR001：
対応中の
チケット

C001：
問い合わせ元
の顧客

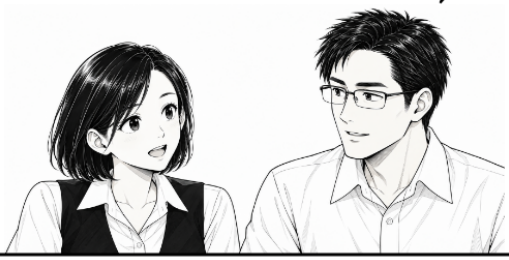
DEV001：
その顧客の機器

AX-100：
機器の機種

この顧客に案内できる文書の範囲、機種、公開状態で絞ってから、距離順に探すんですね。

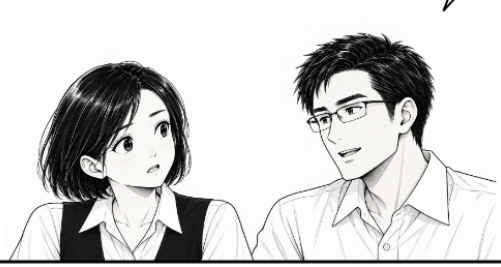
でも、LLMが作るSQLで、顧客の条件を付け忘れたら？

対応中の顧客だけを返すビューを使う。条件を実行側にも持たせるんだ。



その「対応中の顧客」は、DBへどう伝えるんですか？

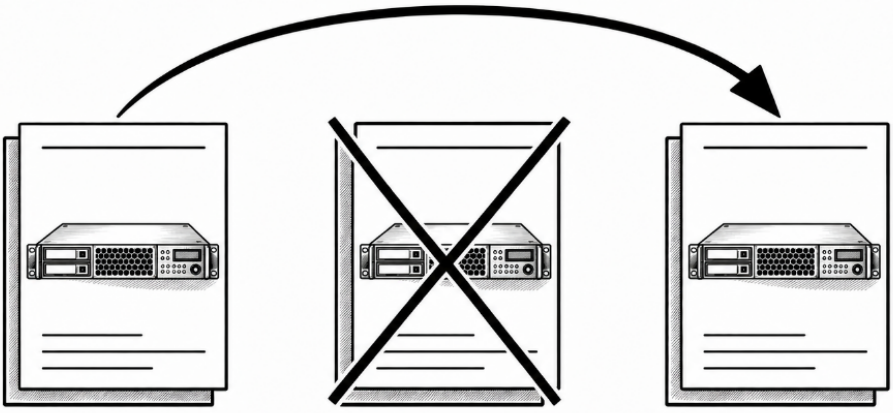
チケットから調べた顧客IDを、このDB接続の中に保持する。



この例では、OJ41_CASE_CTXという
PL/SQLパッケージが値を保持する。

customer_id()で読み出し、ビュー
のWHEREで顧客IDと比べるんだ。

保持した顧客IDをWHERE条件に使う



候補：
C001 /
DEV001

候補：
C002 /
DEV009

返却：
C001 /
DEV001

```
CREATE VIEW OJ41_VISIBLE_DEVICES
AS
SELECT d.*
FROM OJ41_DEVICES d
WHERE d.customer_id =
OJ41_CASE_CTX.customer_id();
```

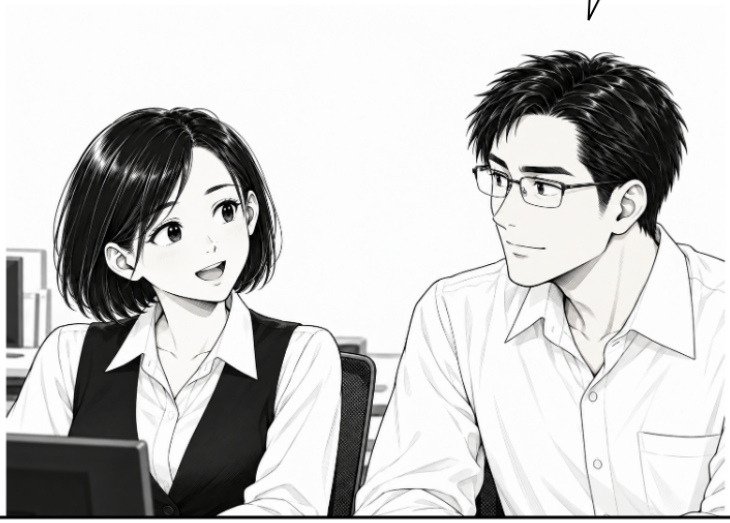
SR001を開くと、この接続にC001を保持。

customer_id()がC001を返す。

WHEREでC001の行だけを返す。

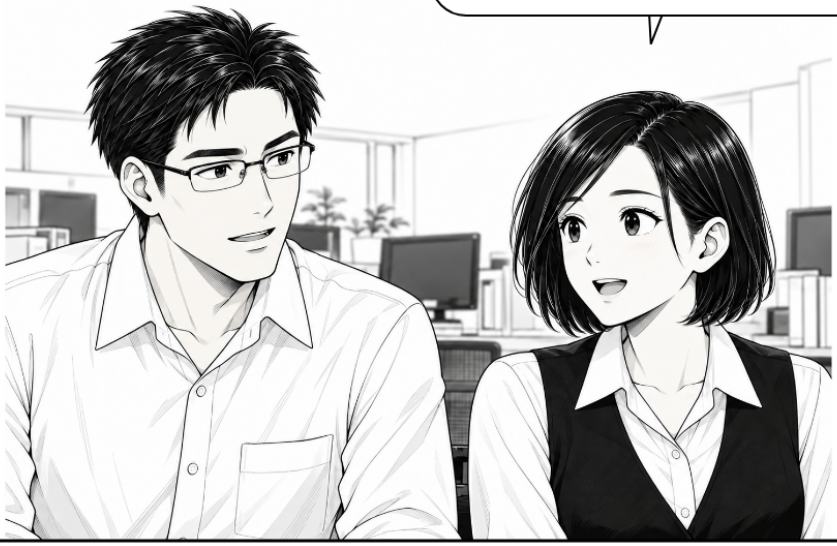
保持するだけではなく、その値をWHEREで使うから、返る行が絞られるんですね。

未設定ならNULLなので一致する行はなく、0件になる。



文書のビューは、公開済みで、共通か対応中の顧客専用のものに絞る。

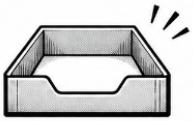
さらに今回の機種で検索すれば、別顧客・別機種の混在を防げますね。



文書を回答に使える範囲

公開済み：共通文書、C001専用文書

除外：C002専用、社内限定、下書き、廃止済み



チケット未設定なら0件

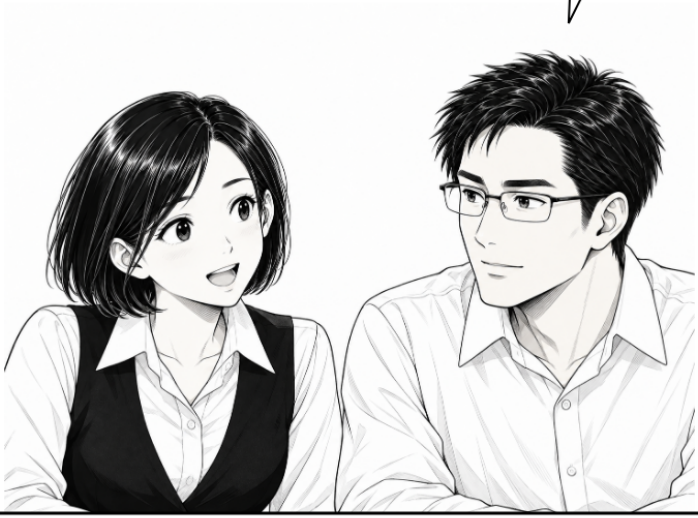
残った候補を機種で絞り、ベクトル距離で比較

ビューなら、AIが顧客条件を忘れても大丈夫ですか？

参照先とDB権限をそのビューに限定する。元の表を直接読めるままでは守れない。

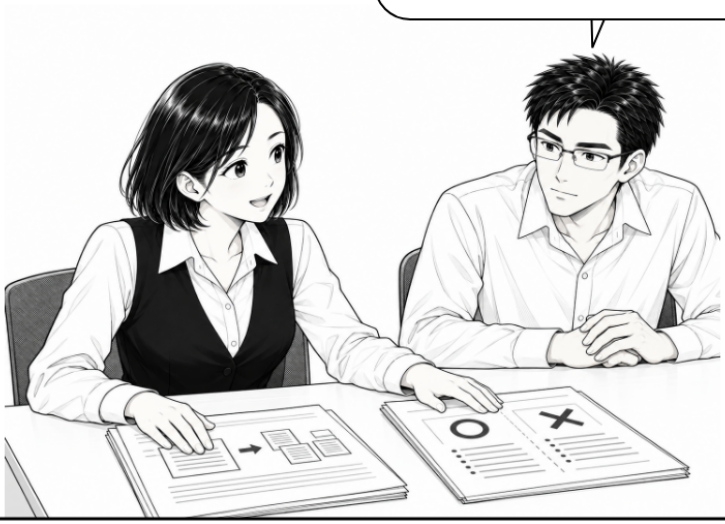
担当者がそのチケットを扱えるかの認可も、アプリ側で確かめます。

そう。LLMに別のチケットや顧客へ勝手に切り替えさせないことも大切だ。

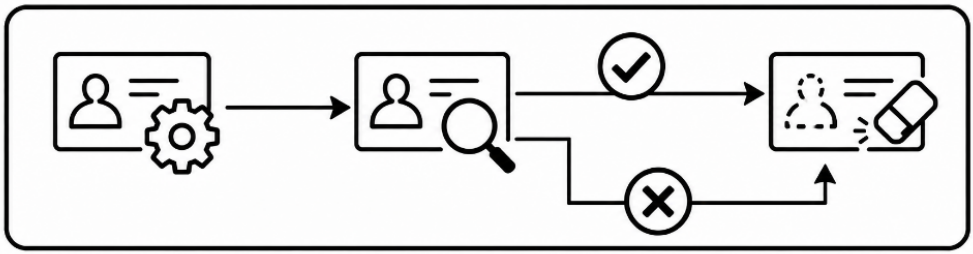


契約が無効なら、文書も出しませんか？

許可された技術文書は案内できる。現地保守の可否とは別だよ。



次の依頼へ顧客情報を残さない



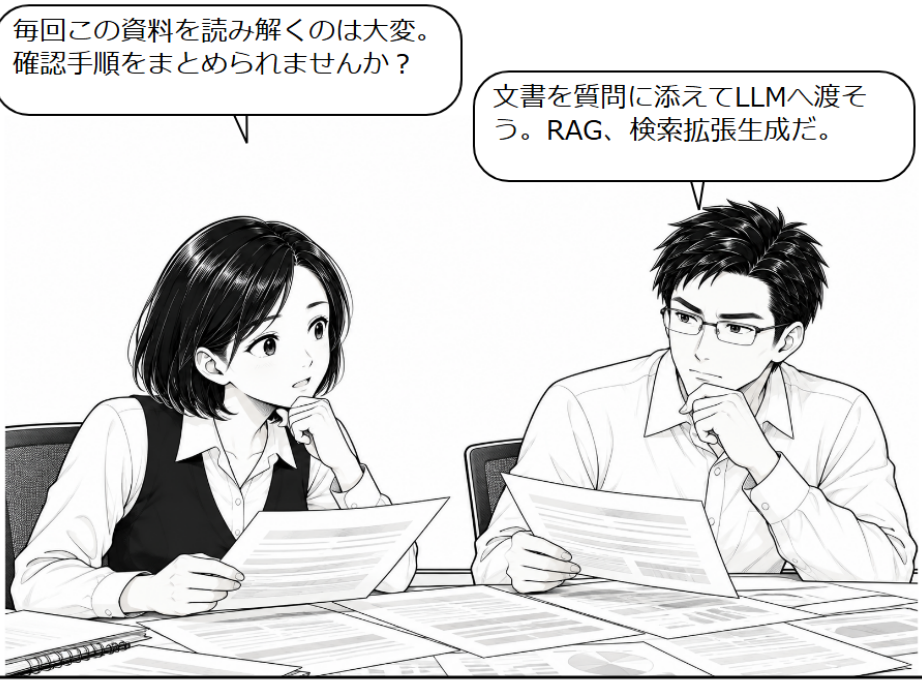
チケットを設定　　同じ接続で照会　　成功・失敗とも消去

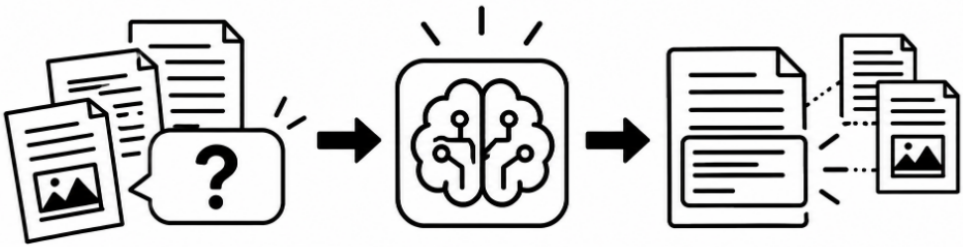
神宮寺
設定から照会までは同じ接続を使い、終わったら必ず消去する。失敗したときもだ。

別顧客の機器ID、別機種、未設定を試します。
認可と接続の後始末も確認します。

文書を探す設計はできました。次は、担当者が使える回答にしたいです。

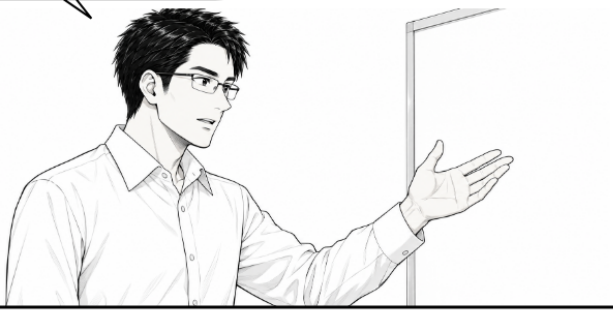
4. 検索した文書を根拠にRAG（検索拡張生成）で回答する





アイ
今回案内できる文書を、回答の根拠にするんですね。

このRAGでは検索・生成・検査の順を
PL/SQLで固定する。
処理選択はAIに任せない。



KB001：電源・LAN・疎通を確認、ログを保存

神宮寺
影響を確かめずに再起動しない、という注意も大切だね。

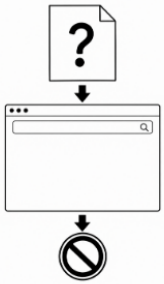
右の用紙KB012：顧客内の運用責任者へ連絡。
訪問は契約と承認を確認

アイ
取得本文と、回答の文書ID・引用を照合します。

引用が原文にあっても、回答の意味まで正しいとは限りませんね。

そう。無関係な引用や、本文を超えた断定を確かめよう。根拠がない場合もね。

根拠不足①：候補がない



神宮寺
記事のこの例はE999の文字列でも絞ると候補ゼロ。LLMを呼ばず、根拠不足を返す。

根拠不足②：文書はあるが、保証延長の金額がない

神宮寺
検索できても、質問への答えが載っているとは限らない。

分からない答えは作らない。候補ゼロと、答えの記載なしを別々に試します。

固定順のRAGを採用します。検索の成否と、回答が根拠に沿うかを分けて評価します。

残った「現地保守を頼めるか」は？

契約ですね。自然な言葉で、対象機器の一覧も聞けるようにしたいです。

5. NL2SQL で契約情報を照会する

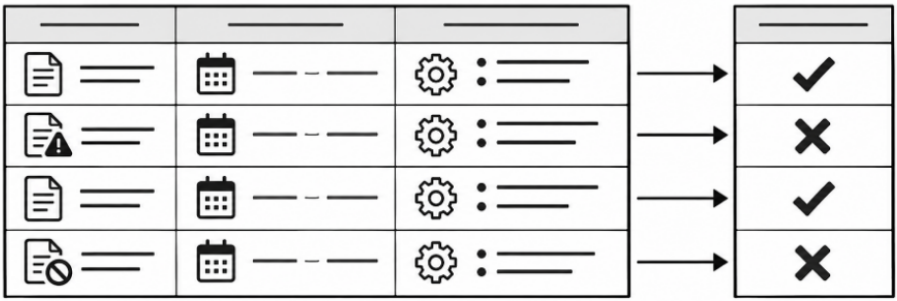
「この顧客の現地保守対象機器を一覧にして」のような質問ができるようにしたいです。
NL2SQLの出番ですか？

Oracleには、Select AIという
NL2SQL機能があるから検討してみよう。

現地保守の可否は、先ほどのビューの判定列を使えますね。

そう。三条件をLLMに考え直させず、判定済みの列を使おう。

自然言語の意味を、判定済みの列へ

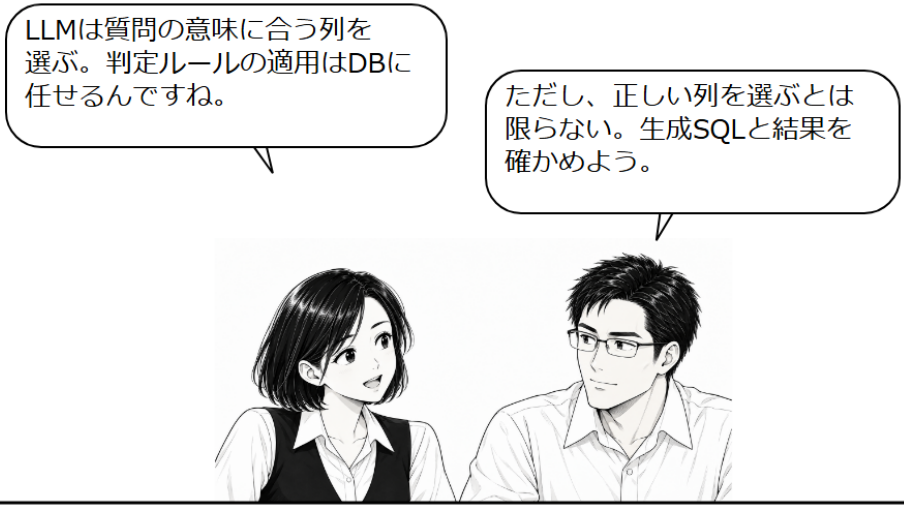


```
SELECT device_id
FROM OJ41_VISIBLE_CONTRACTS
WHERE onsite_eligible = 'Y';
```

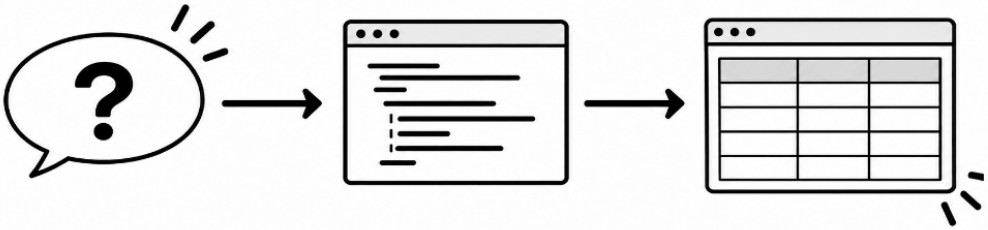
質問：「この顧客の現地保守対象機器を一覧にして」

このビューは顧客の範囲と、CASEの判定列をまとめたもの。

このSQLは期待する形。生成結果を確認する。



アイの試作テスト



自然言語の質問

SHOWSQLで生成
SQLを表示

確認して実行・
結果照合

神宮寺
まずSQLを見る。その後、実行した結果を正解と比べよう。

参照先を制限する前の試作

あれ、違う表を検索しています。名前だけでは、
表の意味が伝わらないんですね。

顧客のWHERE条件も誤り

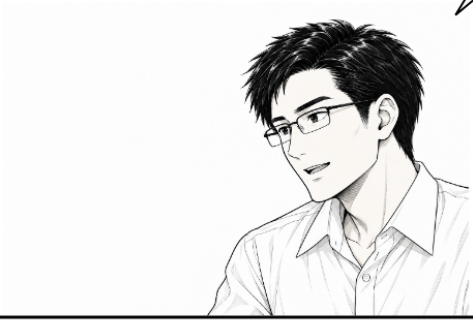
表を直して再試行したら、
今度は別の顧客の機器が混ざりました。
WHERE句の条件が違います。

Select AIには、参照先の制限や注釈な
ど、生成を改善する手段がある。

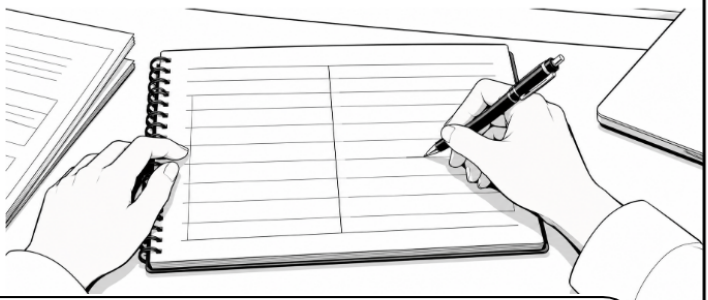
さっきのビューと組み合わせよう。顧客の範囲と
契約判定はDBで守るんだ。

許可したビューだけを使う設定にします。列の意味も伝えられますか？

列の意味は注釈で伝えられる。直したSQLを例にするFeedbackも使える。



期待する結果と照合



アイ
対象顧客だけになりました。期限切れと現地保守なしも対象外。今回のテストは期待どおりです。

注釈は権限の代わりにはならない。ビューの実行権限と、担当者の認可もそろえよう。

自然な言葉の質問と、DBで守る条件。役割を分けて設計できました。
では、手順と契約を一度に聞かれたら？

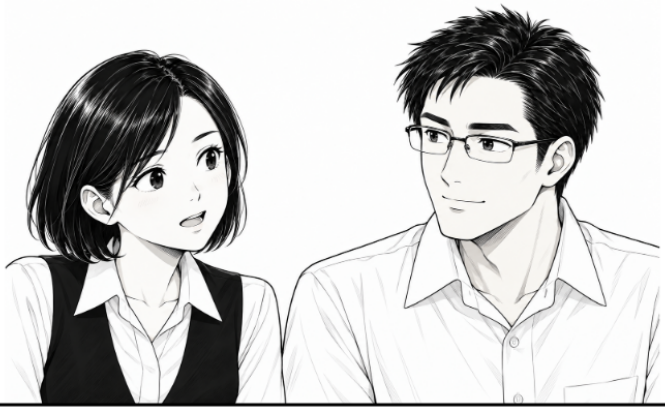
6. 文書検索と契約照会をAgentのツールにする

手順だけ、契約だけ、両方。依頼に合わせて処理を選びたいです。

Agentなら、目的に応じてLLMが道具を選び、結果を見て次の処理を考える。

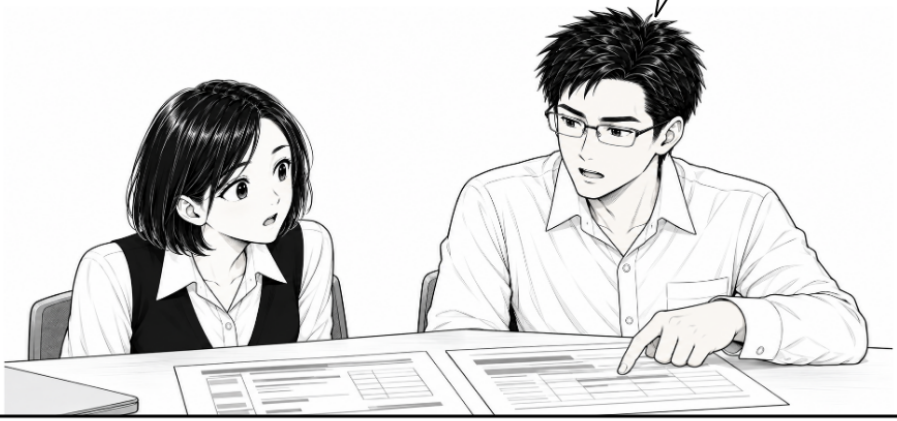
道具って、システムに用意する処理のことですか？

そう。Agentが呼び出せる形にした処理を、ツールと呼ぶ。

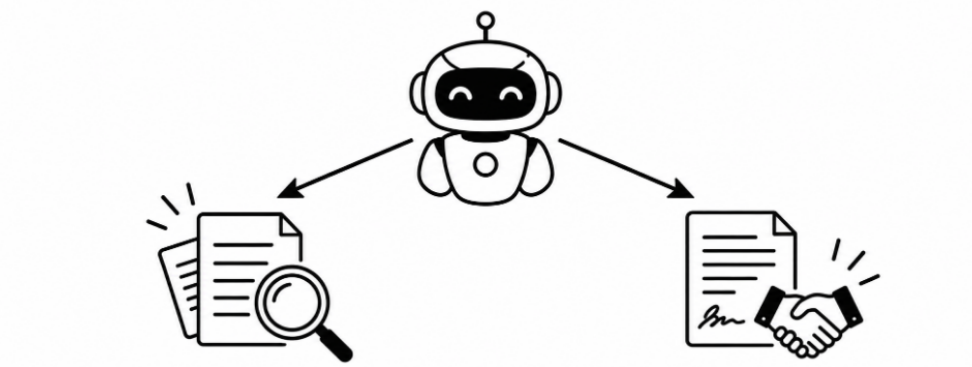


NL2SQLとの関係も整理したいです。

まずはAgentを理解するため、固定SQLの二つのツールで試そう。



固定SQLを実行する二つのツール



文書検索：条件で絞って距離順

契約照会：ビューの判定列を読む

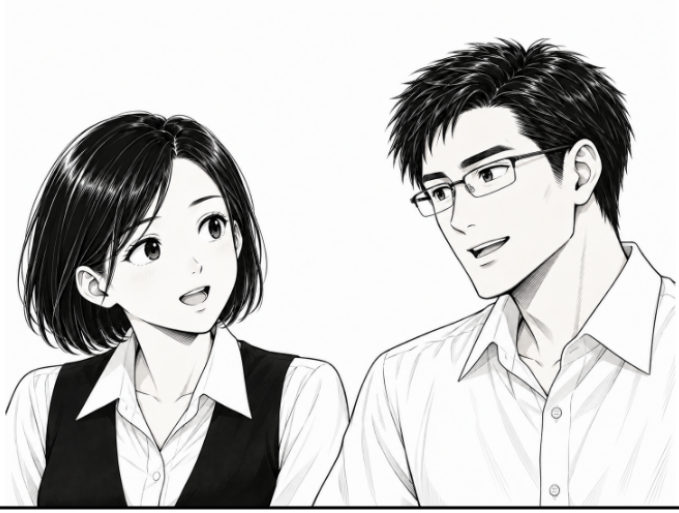
神宮寺

ここでは毎回SQLを生成しない。
道具をどう呼び、結果を使うかに集中しよう。

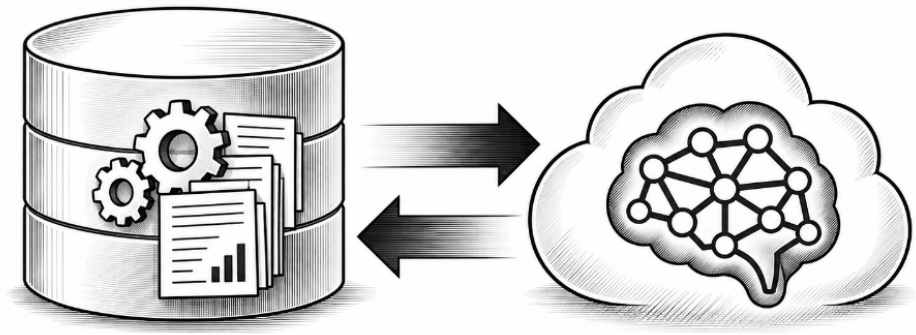
なるほど。先ほどのNL2SQLと切り分けると、Agentの働きが見えますね。

このAgentを動かす基盤は、どこに置くんですか？

DB内でAgentを実行できるSelect AI Agentを使ってみよう。



Select AI Agentの実行基盤

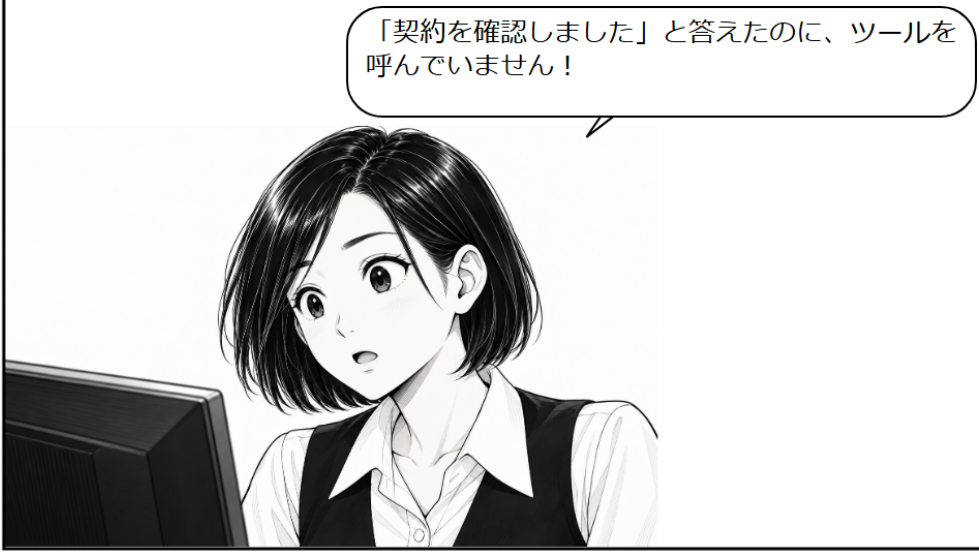


DB：ツール実行と履歴

クラウド：判断するLLM

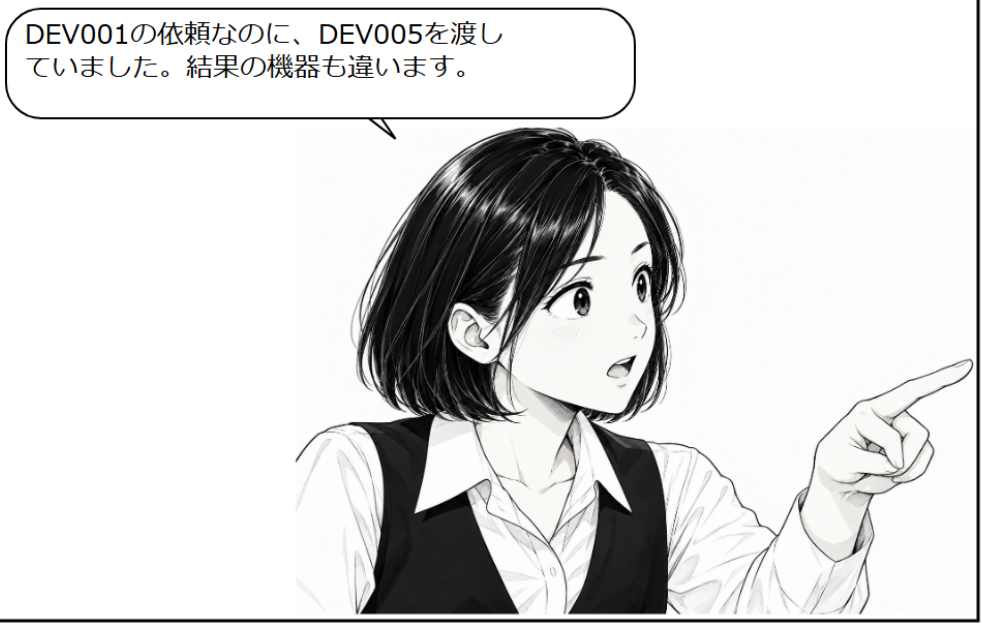
アイ
基盤はDBの中。
LLMへ渡す指示・ツール定義・結果の内容も設計します。

検査を入れる前の試作：未呼出し



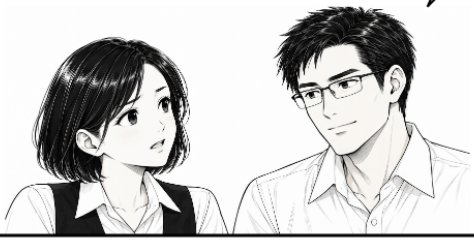
失敗例②：実行エラーを成功扱い

失敗例③：呼び出しの引数違い



LLMに「実行を確認して」と、もう一度頼めばいいですか？

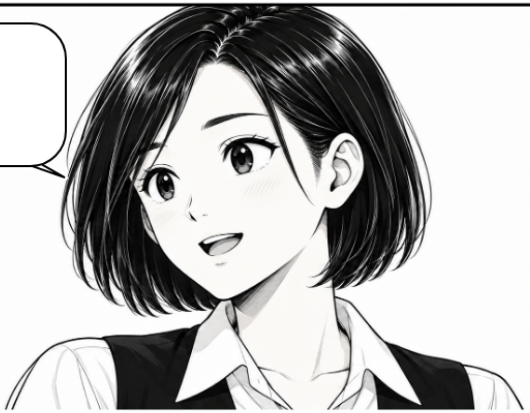
その確認も間違えることがある。ここもハーネスの仕事だよ。



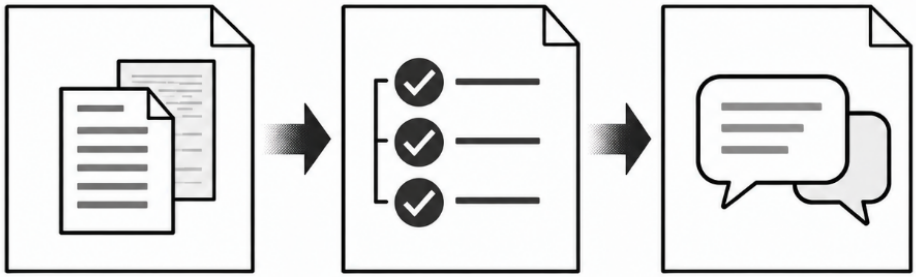
今のチケットをコードで保証する

担当権限を確認した今のチケットから、対象機器をアプリで設定する。不一致はDBで止める。

検索語はLLMに任せても、チケットと対象は勝手に変えさせないんですね。

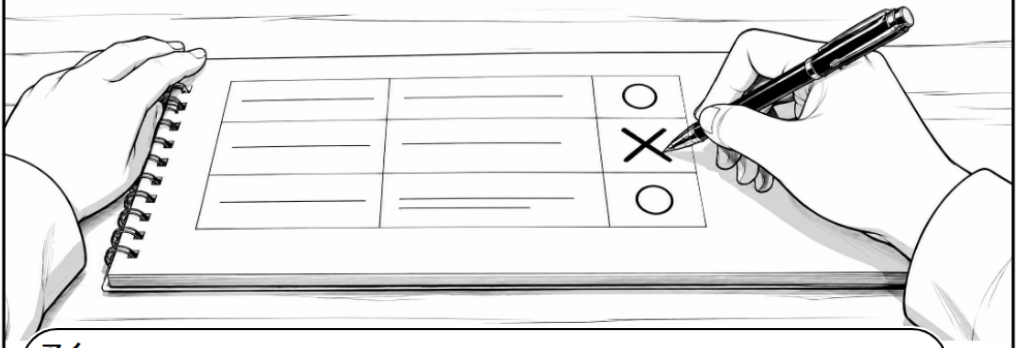


ツールの履歴で実行を確かめる



今回の入力と対象　成功した返却内容　回答との照合

神宮寺
Select AI Agentにはツールの呼出履歴を参照するビューがある。記録と返却内容をコードで確かめよう。



アイ
呼出しなし・実行失敗・対象違いなら、成功した回答として通さない。引用や契約値を照合する検査も用意しました。

検査を加えて、もう一度

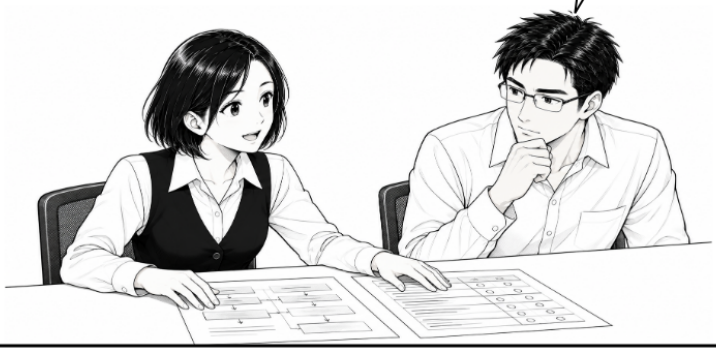
KB001：電源・LAN・疎通の確認とログ保存

二つのツールを実行し、
対象も結果も一致しました。
再起動前の影響確認も、回答に残っています。

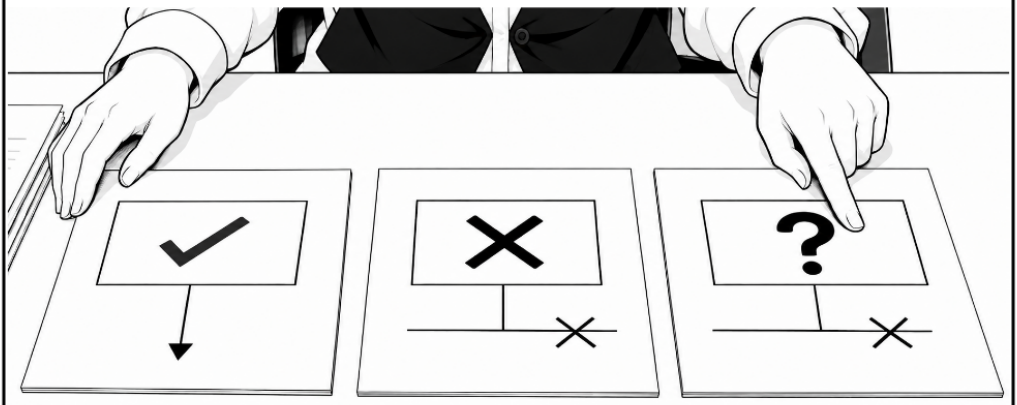
KB012：顧客内の運用責任者へ連絡

現地保守の対象でも、訪問の確約には担当者の承認が必要です。

対象・承認済み・予約済みは別。画面でも区別しよう。



契約判定と訪問承認を分ける



対象：
承認を確認

対象外：
承認へ進めない

不明：
承認へ進めない

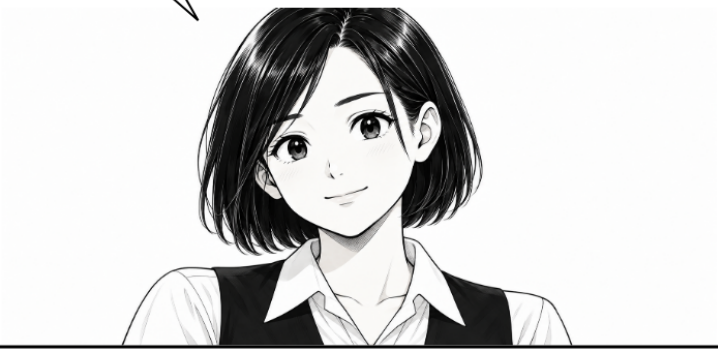
アイ
技術手順は閲覧が許可されていれば案内する。ここも分けて検証します。

LLMとハーネスの役割分担

意味を理解するLLMと、業務判断を一貫して適用するハーネス。二つのツールで役割がつながりました。

今度は要件に合わせて、必要なツールと、その守り方を設計できるね。

はい。使えるデータ、引数、失敗時の扱いまで含めて。Agentアプリを設計できそうです！



次は、要件に合ったツール群へ

コードと環境準備は原記事のAppendixへ。